

Тема № 12. Использование объектов своих классов в контейнерах библиотеки STL

12.1. Требования к классам, объекты которых являются элементами последовательных контейнеров. Сортировка объектов контейнера

Пусть `A`, класс созданный программистом. Особенности, которые необходимо учитывать:

1. Форма записи `vector<A> vec(4);`

Требует в классе `A` конструктор без параметров, при его отсутствии будет ошибка.

2. В коде `A a; vec.push_back(a);`

В `push_back` вызывается конструктор копирования, если требуется «глубокое» копирование, например, память под данные в классе выделяется динамически (в «куче»), то этот конструктор необходимо переопределить.

3. В коде `vec.push_back(A());`

Вызывается конструктор перемещения (перемещается в вектор созданный временный объект), если в классе перемещение отличается от копирования, например, данные находятся в «куче» (в динамической памяти), то следует учитывать, что перемещение работает быстрее, желательно задать конструктор перемещения (иначе, будет вызываться конструктор копирования). Кроме того, при добавлении новых объектов в контейнер-вектор, вектор периодически реаллоцируется (увеличивает размер выделенной памяти), при этом используется конструктор копирования.

Сортировка контейнера

При сортировке можно использовать функцию-предикат (см. тему 7, 1-ый семестр), которая передается в глобальную функцию (алгоритм) `sort` по указателю (для `list` используется метод класса `sort`) или передавать в качестве параметра лямбда-функцию (выражение).

Есть алгоритм `sort` с двумя параметрами: итератор на первый элемент внутри контейнера и на последний элемент внутри контейнера, между этими элементами выполняется сортировка элементов (по аналогии, в `list` есть метод `sort` без параметров). При использовании этого алгоритма нужно иметь правило сравнения двух объектов класса между собой, для этого необходимо выполнить перегрузку операции `<`. Ниже приведен пример подобной сортировки:

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
using namespace std;
class A
{
    string str;
    int value;
public:
    A(string str, int value) : str(str), value(value) {}
    bool operator<(const A & ob2) // Для сортировки надо уметь сравнивать 2 объекта
    {
```

```

        return str < ob2.str; // Сортировка по строковому полю
    }
    friend ostream& operator<<(ostream& out, const A & ob);
};

ostream& operator<<(ostream& out, const A & ob)
{
    out << "str=" << ob.str << " value=" << ob.value;
    return out;
}

int main()
{
    vector<A> vec;
    vec.push_back(A("World", 127));
    vec.push_back(A("Hello", 8));
    vec.push_back(A("Apple", 100));
    sort(vec.begin(), vec.end()); // Вызываем алгоритм для сортировки
    for (auto &pos : vec) cout << pos << endl;
    return 0;
}

```

12.2. Использование лямбда-функций в алгоритмах

В стандарте C++11 появились лямбда-выражения или лямбда-функции, позволяющие создавать определения подставляемых функций, которые можно использовать в качестве параметра или локального объекта.

Лямбда-выражение – это определение анонимного объекта-функции непосредственно в месте его вызова или передачи в функцию в качестве аргумента.

Минимальная лямбда-функция не имеет параметров и просто делает что-то. Например:

```

[] {
    std::cout << "hello lambda" << std::endl;
}

```

Эту функцию можно вызывать непосредственно

```

[] {
    std::cout << "hello lambda" << std::endl; } ();
или передавать вызываемым объектам
auto I = [] {
    std::cout << "hello lambda" << std::endl;
};
I(); // выводим на экран "hello lambda"

```

Лямбда-функция с параметрами

```

auto I2 = [] (std::string str){
    std::cout << str << std::endl;
};

```

```

I2("Hello World!!!!");

```

Захваты (доступ к внешней области видимости)

В инициаторе лямбда-функции (квадратных скобках перед лямбда-функцией) можно задать список захвата (capture) для доступа к данным из внешней области видимости, которые не передаются как аргументы.

- Символы [=] означают, что внешняя область видимости передается в лямбда-функцию по значению.
- Символы [&] означают, что внешняя область видимости передается в лямбда-функцию по ссылке.

Для каждого объекта в лямбда-функции необходимо указать режим доступа к нему: по значению или по ссылке. Это позволяет ограничивать доступ и смешивать разные режимы.

```
int x = 0;      int y = 42;
auto qq = [x, &y] {
std::cout << "x:  " << x << std::endl;
std::cout << "y:  " << y << std::endl;
++y; // OK
};
x = y = 77;
qq(); qq();
std::cout << "final y:  " << y << std::endl;
у будет изменяться, x нет.
```

Пример:

```
#include <iostream>
#include <string>
int main(int argc, char* argv[])
{
    [] {
        std::cout << "hello lambda" << std::endl; } ();
        auto I = [] {
            std::cout << "hello lambda" << std::endl;
        };

        I();

        auto I2 = [](std::string str) {
            std::cout << str << std::endl;
        };

        I2("Hello World!!!!");
        int x = 0;      int y = 42;
        auto qq = [x, &y] {
            std::cout << "x:  " << x << std::endl;
            std::cout << "y:  " << y << std::endl;
            ++y; // OK
            // ++x; Ошибка изменять нельзя
        };
        x = y = 77;
        qq(); qq();
        std::cout << "final y:  " << y << std::endl;
        return 0;
    }
}
```

Использование лямбда-функций в STL

```
// Возводим все элементы в куб
transform(coll.begin(), coll.end(),
coll2.begin(),
[](int d) // Лябда как функтор
```

```

{    return d*d*d;    });
// Ищем первый элемент > x и < y
auto pos = find_if(coll.cbegin(), coll.cend(), // range
[=](int i) {    // Лямбда функция вместо предиката
return    i > x && i < y;    });
Сортировка с использованием лямбда-функций
class Person { public:    string firstname() const;    string lastname() const;    //...};
int main(){    deque<Person> coll;
//...    Ввод элементов для дека
// sort
sort(coll.begin(),coll.end(),                // range
[] (const Person& p1, const Person& p2) { // sort
return p1.lastname()<p2.lastname() ||    (p1.lastname()==p2.lastname() &&
p1.firstname()<p2.firstname());        });    //...}

```

Пример:

```

#include <iterator>
#include <algorithm>
#include <deque>
#include <vector>
#include <iostream>
using namespace std;

int main()
{
    deque<int> coll = { 1, 3, 19, 5, 13, 7, 11, 2, 17 };

    vector<int> coll2;
    coll2.resize(coll.size());
    // Возводим все элементы в куб
    transform(coll.begin(), coll.end(),
        coll2.begin(),
        [](int d) // Лямбда как функтор
        {
            return d*d*d;
        });

    copy(coll2.cbegin(), coll2.cend(),                // source
        ostream_iterator<int>(cout, " ")); // destination
    cout << endl;
    int x = 5;
    int y = 12;
    // Ищем первый элемент > x и < y
    auto pos = find_if(coll.cbegin(), coll.cend(), // range
        [=](int i) { // Лямбда функция вместо предиката, знак = означает все (x и y)
        передается по значению
            return i > x && i < y;
        });
    cout << "first elem >5 and <12: " << *pos << endl;
}

```

12.3. Функциональные объекты (функторы)

Функциональный объект это то, что ведет себя как функция, можно вызывать с помощью пары скобок и передавать параметры.

Примеры:

- указатели на функции;

- объекты классов (функторы), содержащих перегруженную операцию operator() (в узком смысле имеется виду именно это понятие, особенность в том, что существует внутреннее состояние);
- лямбда-функции.

Стандартные функциональные объекты

Критерий сортировки с помощью операции < представляет собой стандартный функциональный объект less<>, с помощью операции > функциональный объект greater<>

```
set<int, less<int>> coll; // Сортировка с помощью <
set<int, greater<int>> coll; // Сортировка >
transform(coll.cbegin(), coll.cend(), coll.begin(),
negate<int>());           // Изменение знака элемента
```

Умножение элементов 2-х контейнеров:

```
transform(coll.cbegin(), coll.cend(), // 1-й источник
coll.cbegin(),                       // 2-й источник
coll.begin(),                        // Назначение
multiplies<int>());                  // Умножение
```

Пример:

```
#include <iostream>
#include <iterator>
#include <set>
#include <deque>
#include <algorithm>
using namespace std;

int main()
{
    deque<int> coll = { 1, 2, 3, 5, 7, 11, 13, 17, 19 };

    cout << "initialized: ";
    copy(coll.cbegin(), coll.cend(),
        ostream_iterator<int>(cout, " "));

    // negate all values in coll
    transform(coll.cbegin(), coll.cend(),
        coll.begin(),
        negate<int>());           // Изменение знака элемента

    cout << endl << "negated: ";
    copy(coll.cbegin(), coll.cend(),
        ostream_iterator<int>(cout, " "));
    // square all values in coll
    transform(coll.cbegin(), coll.cend(), // Первый источник
        coll.cbegin(), // Второй источник
        coll.begin(), // Назначение
        multiplies<int>()); // Умножение

    cout << endl << "squared: ";
    copy(coll.cbegin(), coll.cend(),
        ostream_iterator<int>(cout, " "));

    set<int, less<int>> coll2 = { 10, 30, 2, 6, 7, 8, 1 }; // Сортировка с помощью <
    cout << endl << "coll2: ";
```

```

        copy(coll2.cbegin(), coll2.cend(),
              ostream_iterator<int>(cout, " "));
        set<int, greater<int>> coll3 = { 123, 34, 56, 37, 18, 2, 3, 5, 7 }; // Сортировка
с помощью >
        cout << endl << "coll3: ";
        copy(coll3.cbegin(), coll3.cend(),
              ostream_iterator<int>(cout, " "));
        return 0;
}

```

Пример созданного функционального объекта:

```

#include <list>
#include <algorithm>
#include <iostream>
#include <iterator>

using namespace std;

// Функциональный объект добавляющий к элементу значение, полученное при инициализации
class AddValue {
private:
    int theValue;    // Значение
public:
    // Конструктор
    AddValue(int v) : theValue(v) {
    }

    // Вызов функции для элемента для которого добавляется значение
    void operator() (int& elem) const {
        elem += theValue;
    }
};

int main()
{
    list<int> coll;

    // Добавляем элементы в список
    for (int i = 1; i <= 9; ++i) {
        coll.push_back(i);
    }

    cout << "initialized: ";
    copy(coll.cbegin(), coll.cend(), // source
          ostream_iterator<int>(cout, " "));

    // Добавляем значение 10 к каждому элементу
    for_each(coll.begin(), coll.end(), // диапазон
              AddValue(10));           // операция

    cout << endl << "after adding 10: ";
    copy(coll.cbegin(), coll.cend(), // source
          ostream_iterator<int>(cout, " "));

    // Добавляем значение первого элемента к каждому элементу
    for_each(coll.begin(), coll.end(), // range
              AddValue(*coll.begin())); // operation
    cout << endl << "after adding first element: ";
    copy(coll.cbegin(), coll.cend(), // source

```

```

        ostream_iterator<int>(cout, " ");
    }
}

```

12.4. Требования к классам, объекты которых являются элементами контейнеров *set* и *map*

Контейнеры *set* и *map* являются отсортированными, хранятся в памяти в виде двоичного дерева поиска (красно-черное дерево). Поэтому должен быть механизм сравнения любых двух элементов между собой, чтобы это обеспечить, для объектов класса должна быть перегружена операция `<` (меньше, по умолчанию). Пример представлен ниже.

```

#include <iostream>
#include <string>
#include <set>
using namespace std;
class A
{
    string str;
    int value;
public:
    A(string str, int value) : str(str), value(value) {}
    bool operator<(const A & ob2) const // Для сортировки надо уметь сравнивать 2 объекта
    {
        return str < ob2.str; // Сортировка по строковому полю
    }
    friend ostream& operator<<(ostream& out, const A & ob);
};

ostream& operator<<(ostream& out, const A & ob)
{
    out << "str=" << ob.str << " value=" << ob.value;
    return out;
}

int main()
{
    set<A> s;

    s.insert(A("World", 127));
    s.insert(A("Hello", 8));
    s.insert(A("Apple", 100));

    for (auto &pos : s) cout << pos << endl;
    return 0;
}

```

12.5. Требования к классам, объекты которых являются элементами контейнеров *unordered_set* и *unordered_map*

Объекты внутри этих контейнеров в памяти хранятся в виде хеш-таблицы, поэтому

необходимо для объектов используемого класса задать хеш-функцию («хешер») и перегрузить операцию == (равно) для объектов класса. Для стандартных типов для получения хеша рекомендуется использовать шаблонную структуру hash, объект шаблона hash является функтором. Например, hash<double> dhash; double x; dhash(x); // Получение хеша для переменной x типа double.

Если класс (структура) имеет несколько полей разных типов, то для получения комбинированного хеша, рекомендуется делать поступать так, как показано в примере ниже (задается класс-«хешер», объект класса является функтором):

```
#include <iostream>
#include <string>
#include <unordered_set>
using namespace std;
struct Person {
    string name; // Имя
    int age; // Возраст
    double weight; // Вес
    bool operator==(const Person & ob2) const
    {
        return name == ob2.name && age == ob2.age && weight == ob2.weight;
    }
};

// Хешер для Person
struct PersonHasher
{
    // Хешеры для отдельных полей
    hash<string> shash;
    hash<int> ihash;
    hash<double> dhash;
    size_t operator()(const Person & ob) const
    {
        const size_t coef = 2'946'901; // 5171;// 11171;// 103141;// 2'946'901;
        // Получаем комбинированный хеш
        return (
            coef * coef * shash(ob.name) +
            coef * ihash(ob.age) +
            dhash(ob.weight)
        );
    }
};

int main()
{
    unordered_set<Person, PersonHasher> un_set;
    un_set.insert({ "Ivanov", 35, 73.5 });
    un_set.insert({ "Petrov", 30, 70.0 });
    un_set.insert({ "Sidorov", 29, 90.0 });
    return 1;
}
```

Получение «хорошего» хеша, который равномерно раскладывает ключи по диапазону хеш-таблицы с целью уменьшения коллизий (когда два или более разных элементов имеют один хеш), отдельная задача. В качестве множителя coef рекомендуется использовать простые числа. В зависимости от числа используемых полей для получения общего хеша,

рекомендуется использовать числа: 5171; 11171; 103141; или 2'946'901;